FitBuddy-AI Fitness Plan Generator Using Gemini Models

Government Arts and Science College-Manur-MSU142

TeamLeader:

Sandhiya P NM ID:F24D0CD882B6C7E9B9F44821FF74E70B

Team Members:Santhanamari G NM ID: C1A1D3256DDD23E87C0472B52EDBF20B

Saraswathi P NM ID: F7D8EF3617B74F0922A414AB1EDEF80E

Chinnathai S NM ID:6A593ADCC611910BA8240846346DA660

Muthumari M NM ID: 61F719397D001D3BE07494880A4743B7

Project Development Phase

Here's the *Project Development Phase* for FitBuddy – AI Fitnes Plan Generator, showing how the design is translated into working components:

🛠️ Project Development Phase: FitBuddy

1. Backend Development

- *FastAPI Setup*
  - Entry point: main.py initializes the app.
  - Routing logic: routes.py handles user input, feedback, and admin views.
- *Core Modules*
  - gemini_generator.py → generates 7-day workout plans using Gemini Pro.
  - gemini_flash_generator.py → produces nutrition/recovery tips via Gemini Flash.
  - updated_plan.py → revises workout plans based on user feedback.
- *Database Integration*
  - database.py with SQLAlchemy ORM.
  - Tables: User (profile info), WorkoutPlan (original + updated plans).

- Functions: save_user(), save_plan(), update_plan(), get_user().

---

2. Frontend Development

- *Templates*
  - index.html → user input form (name, age, weight, goal, intensity).
  - result.html → displays workout plan + nutrition tip + feedback form.
  - all_users.html → admin dashboard with user details and plans.
- *UI Design*
  - Responsive layout using CSS + Flexbox.
  - Gym-themed background image for motivation.
  - Mobile-friendly with media queries.
- *Dynamic Rendering*
  - Jinja2 templates bind backend data to frontend.
  - Example: {{ workout_plan }} and {{ nutrition_tip }} displayed after submission.

---

3. AI Integration

- *Workout Plan Generation*
  - Prompt structured for Gemini Pro to return day-wise plans.
  - Includes warm-up, main workout, cooldown.
- *Nutrition Tip Generation*
  - Gemini Flash provides concise, practical tips.
- *Feedback Loop*
  - User feedback passed to Gemini Pro → updated plan generated.
  - Both versions stored in DB for comparison.

---

4. Testing & Validation

- *Unit Testing*
  - Validate AI functions (generate_workout_gemini, generate_nutrition_tip_with_flash).
  - Ensure DB operations (save, update, retrieve) work correctly.
- *Integration Testing*
  - Test full workflow: input → AI generation → DB storage → frontend rendering.
- *Error Handling*
  - Catch API failures and return user-friendly messages.
  - Validate inputs with Pydantic schemas.

---

5. Deployment

- *Local Deployment*
  - Run with Uvicorn: uvicorn app.main:app --reload.
  - Access via http://127.0.0.1:8000.
- *API Documentation*
  - Auto-generated docs at /docs.
- *Future Deployment*
  - Cloud-ready architecture for scaling.
  - Potential Docker/Kubernetes setup.

---

6. Deliverables in Development Phase

- Functional backend with Gemini integration.
- Responsive frontend templates.
- SQLite database storing user and plan data.
- Admin dashboard for trainers.
- Tested and deployed locally with FastAPI + Uvicorn.